



# 손영진

✉ odso4139@naver.com ☎ 010-3975-5037

🔗 portfolio-kohl-xi-2o.vercel.app 🌐 github.com/sonprogrammer

## PROFILE

실시간 데이터와 서비스 운영 흐름을 사용자 경험으로 연결해 온 프론트엔드 개발자 손영진입니다.

Next.js와 TypeScript를 기반으로 O2O 서비스, 관리자 대시보드, AI 연동 서비스 등을 직접 설계하고 구현했습니다. MungPass에서는 QR 기반 입장과 Supabase Realtime을 활용한 이용 현황 동기화, 역할별 권한 관리, 매출 대시보드를 구축했으며, Fuelly에서는 AI 응답을 서비스에서 활용 가능한 JSON 데이터로 구조화했습니다.

기능 구현 이후에도 데이터 흐름과 상태 관리, 성능을 점검하며 실제 사용 과정에서 발생하는 문제를 개선하는데 집중하고 있습니다.

현재는 RAG 설계를 적용하며 관련 구조를 학습하고 Codex 기반 AI 개발 워크플로우를 직접 구현·적용해 AI를 서비스 기능과 개발 과정에 활용하고 있습니다.

## SKILLS

### Back-end

- Node.js
- Express.js
- MongoDB
- Supabase

### AI / Development Workflow

- RAG
- Gemini API / Groq API
- Codex
- AI Agent Workflow

### Front-end

- JavaScript
- TypeScript
- React
- Next.js
- TanStack-Query
- Zustand
- Recoil
- REST API
- WebSocket
- Axios
- Tailwind CSS

## PROJECTS

### 애견카페 통합 O2O 서비스 및 B2B 관리 시스템 | MungPass

01/2026 – 05/2026

개인 프로젝트 · 1인 기획/개발

애견카페 이용자와 매장 운영자를 연결하고 입장부터 이용 현황·매출 관리까지 하나의 서비스에서 처리할 수 있도록 설계한 O2O 서비스입니다. Next.js, TypeScript, Supabase를 기반으로 서비스 구조와 주요 기능을 직접 구현했습니다.

#### [실시간 데이터] QR 입장 및 이용 현황 동기화

- 매장별 QR을 활용한 입장 기능을 구현하고 Supabase Realtime과 TanStack Query를 연동해 방문·이용 상태가 매장 운영자 화면에 새로고침 없이 반영되도록 구성했습니다.

#### [B2B 대시보드] 운영 데이터 시각화

- Refine과 Recharts를 활용해 일별·월별 매출, 방문자 수, 평균 이용시간 등 매장 운영 지표를 확인할 수 있는 관리자 대시보드를 구현했습니다.

#### [AI 연동] Gemini 기반 매출 분석 및 결과 캐싱

- 매출·방문자 수·객단가 데이터를 Gemini API로 분석해 운영 인사이트를 제공하고 생성 결과를 DB에 저장해 동일 분석 결과의 반복 API 호출을 줄였습니다.

#### [아키텍처·권한] FSD 및 RLS 적용

- 기능 추가 시 변경 범위를 줄이기 위해 FSD 구조를 적용하고, TanStack Query 기반의 데이터 조회·변경 로직을 Custom Hook으로 분리해 컴포넌트의 UI 로직과 서버 상태 관리 책임을 구분했습니다. 또한 Supabase RLS와 라우트 권한 처리를 통해 사용자·매장 운영자·관리자별 데이터 접근 범위를 분리했습니다.

#### [성능·모바일 UX] Lighthouse 및 PWA 개선

- 이미지 로딩과 렌더링 구조를 개선해 LCP 2.0초 이내, CLS 0.001 이하를 기록했으며 Service Worker와 manifest를 적용해 PWA 환경을 구성했습니다.

### AI 기반 개인 맞춤 영양관리 서비스 | Fuelly

12/2025 – 01/2026

개인 프로젝트 · 1인 기획/개발

사용자의 신체 정보와 영양 섭취 데이터를 기반으로 부족한 영양소를 계산하고 AI를 활용해 맞춤형 음식 추천을 제공하는 웹 서비스입니다.

#### [AI 데이터 처리] Groq 응답 구조화

- AI 응답 형식 편차로 인한 파싱 오류를 줄이기 위해 Groq의 JSON Object 응답 형식을 적용하고 Next.js Route Handler에서 응답을 검증·가공해 클라이언트가 일관된 데이터 구조를 사용하도록 구현했습니다.

#### [AI UX] 추론 지연 대응

- AI 응답까지 약 2~3초의 대기 시간이 발생하는 점을 고려해 스켈레톤 UI와 상태 기반 로딩 인터랙션을 적용했습니다.

#### [인증] 토큰 관리 및 재발급 동시성 처리

- Access Token은 메모리, Refresh Token은 HttpOnly Cookie로 관리하고 Axios Interceptor에 토큰 재발급 대기 Queue를 구현해 다중 요청 환경에서 중복 재발급을 제어했습니다.

#### [상태 관리] 서버·UI 상태 책임 분리

- TanStack Query는 서버 데이터와 캐싱을 Zustand는 전역 UI 및 클라이언트 상태를 담당하도록 분리해 상태 관리 책임을 명확하게 구성했습니다.

### 실시간 시세 기반 코인 모의투자 서비스 | Virtual Coin

03/2025 – 04/2025

개인 프로젝트 · 1인 기획/개발

Upbit의 실제 거래 데이터를 활용해 시장 변화를 실시간으로 확인하고 모의투자를 경험할 수 있도록 구현한 웹 서비스입니다.

#### [API 처리] Node.js 프록시 서버 구축

- Upbit API 연동 과정에서 발생한 CORS 및 Origin 기반 요청 제한 문제를 처리하기 위해 Node.js 기반 중계 서버를 구성하고 REST API와 WebSocket 데이터를 서버에서 수신해 클라이언트로 전달하도록 구성했습니다.

#### [실시간 데이터] REST + WebSocket 하이브리드

- REST API로 초기 시세를 우선 조회하고 이후 WebSocket으로 실시간 갱신하는 하이브리드 구조를 적용해 Production 환경 5회 측정 기준 첫 화면 시세 로딩 시간을 4.68초 → 0.9초로 약 81% 단축했습니다.

#### [렌더링 최적화] 실시간 업데이트 제어

- 초당 반복되는 WebSocket 데이터가 과도한 상태 업데이트로 이어지지 않도록 lodash.throttle를 적용해 렌더링 빈도를 제어했습니다.

### 트레이너·회원 PT 관리 및 실시간 소통 서비스 | BNTY

09/2024 – 10/2024

개인 프로젝트 · 1인 기획/개발

트레이너 근무 경험에서 발견한 수강횟수 관리와 수업 일지 기록의 불편을 바탕으로 QR 출석·수강권 관리·실시간 채팅을 하나의 흐름으로 구현한 서비스입니다.

#### [업무 자동화] QR 기반 수업 관리

- QR 스캔 시 출석 처리와 잔여 수강 횟수 차감이 함께 반영되도록 구현해 수강 관리 과정에서 발생할 수 있는 누락 문제를 줄였습니다.

#### [실시간 통신] Socket.IO ⚡ 기반 1:1 채팅

- Socket.IO ⚡ 를 활용해 실시간 채팅과 읽음 상태를 구현하고 한글 IME 입력 과정에서 발생한 중복 전송 문제를 isComposing 상태를 확인해 해결했습니다.

#### [성능 개선] 초기 로딩 최적화

- React.lazy 기반 코드 스플리팅, Suspense 로딩 처리, 서버 DB 조회 병렬화를 적용해 초기 로딩 병목을 개선하고 Lighthouse 5회 측정 기준 LCP를 평균 8.0초에서 1.1초로 약 86% 단축했습니다.

## 플레이리스트 공유 SNS | Mingle

11/2023 - 11/2023

팀 프로젝트 · 5인 · 프론트엔드 담당

React·TypeScript 기반 프론트엔드 개발과 Git을 활용한 협업 프로세스를 학습하고 팀 프로젝트를 수행했습니다

- 공통 Axios 인스턴스의 Authorization Header가 외부 OpenWeatherMap API 요청에도 포함되며 발생한 CORS/Preflight 문제를 인증이 필요 없는 별도 Axios 인스턴스로 분리해 해결
- 백엔드 개발자와 API 명세 변경사항을 조율하며 매일 오전 스크럼을 통해 진행 상황과 이슈를 공유하며 프론트엔드 연동 작업 진행
- Git Flow 기반으로 Milestone·Issue·Pull Request를 관리하고 코드 리뷰를 통해 협업 프로세스와 코드 품질 관리

## AI EXPERIENCE

### [RAG 기반 포트폴리오 및 AI 기능 개발]

- 포트폴리오 데이터를 프로젝트별 Markdown 파일로 분리하고 heading 기준의 의미 단위로 Chunking한 뒤 Gemini Embedding모델과 Supabase pgvector를 활용해 Vector DB를 구성
- 사용자 질문과 유사한 문서를 검색해 LLM Context로 전달하는 RAG 기반 포트폴리오 질의응답 기능을 구현
- Gemini·Groq API를 서비스에 연동하고 JSON 형식의 AI 응답 처리, 결과 캐싱, 응답 지연에 대응한 UI를 구현

### [AI 개발 워크플로우]

- Codex와 tmux, Git Worktree를 활용해 작업 환경을 분리하고 여러 작업을 병렬로 진행하는 개발 방식을 실습
- 작업별 요구사항·책임 범위·검증 기준을 사전에 정의하고 AI가 생성한 변경사항을 직접 코드와 동작 기준으로 확인한 뒤 통합하는 방식으로 실습

### [Context 및 작업 세션 관리]

- 역할과 작업 목적에 맞는 지침과 참고 정보만 제공하도록 Skill과 프로젝트 문서를 분리해 불필요한 Context가 한 세션에 누적되지 않도록 구성
- 기능별 GitHub Issue를 기준으로 작업 범위를 정의하고 각 세션이 해당 Issue를 기준으로 작업하도록 구성해 필요한 Context와 책임 범위를 분리

### [AI 활용 및 검증 원칙]

- AI에게 높은 자유도를 부여하기보다 정확한 Context와 도메인 규칙, 명확한 작업 범위와 검증 기준을 제공해 AI가 수행할 범위를 제어하는 것이 중요하다는 점을 경험
- Type Check, Lint, Test, Build처럼 프로그램적으로 검증 가능한 영역은 자동화된 도구를 우선 활용하고 AI는 요구사항 해석, 대안 탐색, 코드 리뷰처럼 판단이 필요한 영역에 활용
- 멀티 에이전트 환경에서도 개발자가 현재 구현 상태와 변경 흐름을 직접 이해하고 결과를 판단해야 한다는 점을 경험하며, Context·규칙·도구·검증 구조를 설계하는 Harness Engineering의 중요성을 체감

### 자기소개

[데이터의 흐름을 이해하고 서비스 경험으로 연결하는 프론트엔드 개발자]

프론트엔드 개발에서 화면에 보이는 기능과 그 뒤에서 움직이는 데이터의 흐름을 함께 이해하는 것을 중요하게 생각합니다. 사용자의 행동이 어떤 데이터로 전달되고 처리되는지, 그 결과가 다시 어떤 화면과 경험으로 이어지는지를 살피며 서비스를 구현해 왔습니다.

애견카페 통합 O2O 서비스인 MungPass를 직접 기획하고 개발하며 이러한 개발 방식을 구체화했습니다. QR 입장 데이터를 Supabase Realtime과 연동해 매장 이용 현황이 운영 화면에 실시간으로 반영되도록 구현했고, 매출과 방문 데이터를 확인할 수 있는 B2B 대시보드도 구성했습니다. 사용자, 매장 운영자, 관리자에게 필요한 기능과 접근 데이터가 달랐기 때문에 역할별 흐름을 나누고 RLS를 적용해 권한 범위를 설계했습니다. MungPass는 실제 서비스로의 확장 가능성을 염두에 두고 시작한 프로젝트였기 때문에 향후 기능과 도메인이 늘어날 경우 발생할 수 있는 코드 복잡도를 고려해 초기부터 FSD 아키텍처를 적용했습니다. 기능별 책임을 분리하고 레이어 간의 의존 방향을 정해 기능 추가와 변경 시 영향을 받는 범위를 파악하기 쉽도록 구조화했습니다.

다른 프로젝트에서도 데이터와 로직의 책임을 분리하는 방식을 일관되게 적용했습니다. 여러 컴포넌트에서 반복되는 상태와 비즈니스 로직은 Custom Hook으로 분리하고 서버 데이터 조회와 변경 로직은 TanStack Query 기반의 Custom Hook으로 구성해 컴포넌트가 UI 표현에 집중할 수 있도록 했습니다.

Fuely에서는 Groq AI가 JSON Object 형식으로 결과를 반환하도록 구성하고, 서버에서 이를 파싱해 서비스 데이터로 안정적으로 활용할 수 있도록 구현했습니다. Virtual Coin에서는 REST API와 WebSocket을 결합한 데이터 수신 구조로 개선해 첫 화면 시세 로딩 시간을 1초 이내로 단축했습니다.

프로젝트를 진행할 때 기능이 동작하는 것에서 끝내지 않고 데이터가 이동하는 과정과 상태의 책임, 사용자가 기다리는 시간과 화면의 반응까지 확인하며 개선할 지점을 찾습니다. 앞으로도 서비스의 요구사항을 정확하게 이해하고 안정적인 데이터 흐름과 사용자 경험을 구현하는 프론트엔드 개발자로 성장하겠습니다.

### 핵심 프로젝트 및 트러블슈팅 경험

[데이터 흐름의 문제를 찾아 구조와 성능을 개선한 경험]

프로젝트를 진행하면서 기능이 예상대로 동작하지 않거나 반복적인 처리로 비효율이 발생하면 데이터가 생성되고 전달되는 과정을 먼저 확인합니다. 문제의 위치를 찾은 뒤 서비스 흐름에 맞는 해결 방법을 적용하고, 결과를 다시 측정하는 방식으로 개발해 왔습니다.

애견카페 O2O 서비스 MungPass에서는 매출·방문 데이터를 기반으로 Gemini API를 호출해 매장 매출 인사이트를 제공했습니다. 그러나 동일 날짜의 데이터에 대해 반복적으로 분석을 요청할 경우 같은 결과를 생성하기 위해 API 호출과 대기 시간이 계속 발생하는 구조였습니다. 이를 개선하기 위해 이미 생성된 분석 결과가 Supabase DB에 존재하면 기존 결과를 재사용하고 결과가 없는 경우에만 Gemini API를 호출한 뒤 저장하도록 변경했습니다. 이를 통해 불필요한 중복 호출과 사용자의 분석 대기 시간을 줄였습니다.

Fuely에서는 Access Token이 만료된 상태에서 여러 API 요청이 동시에 발생하면 각각의 요청이 Refresh Token을 사용해 토큰 재발급을 시도할 수 있는 문제가 있었습니다. 인증 처리를 Axios Interceptor로 통합하고 재발급이 진행 중일 때 추가 요청을 대기시키는 Queue를 구성했습니다. 재발급이 완료되면 대기 중인 요청을 새로운 Access Token으로 다시 실행하도록 처리해 다중 요청 환경에서도 중복 토큰 재발급이 발생하지 않도록 구성했습니다.

Virtual Coin에서는 브라우저에서 Upbit REST API와 WebSocket을 직접 호출하는 과정에서 CORS와 Origin 기반 요청 제한 문제가 발생했습니다. 이를 해결하기 위해 Node.js 중계 서버를 구성하고 Upbit의 REST API와 WebSocket 데이터를 서버에서 수신해 클라이언트로 전달하도록 요청 구조를 변경했습니다. 이후 실시간 시세를 WebSocket으로만 받아 초기 데이터를 구성하면서 첫 화면의 시세 표시가 늦어지는 문제가 있었습니다. 초기 데이터는 REST API로 먼저 조회하고 이후 WebSocket 데이터로 갱신하는 방식으로 구조를 변경했으며 빈번한 실시간 데이터가 과도한 상태 업데이트로 이어지지 않도록 throttle을 적용했습니다. 그 결과 Production 환경에서 5회 측정 기준 첫 화면 시세 로딩 시간을 평균 4.68초에서 0.9초로 약 81% 단축했습니다.

이러한 경험을 통해 문제를 해결할 때 눈앞의 오류만 처리하기보다 데이터가 어디에서 생성되고 어떤 과정을 거쳐 화면에 도달하는지 확인하는 것이 중요하다는 점을 배웠습니다. 앞으로도 원인을 구체적으로 파악하고 서비스의 흐름에 맞는 방법을 선택하며 안정적으로 기능을 구현해 나가겠습니다.

## 팀워크 및 협업 프로세스

[변경사항을 빠르게 공유하고 함께 해결책을 맞추는 협업]

팀 프로젝트에서는 각자 맡은 기능을 구현하는 과정만큼 서로의 진행 상황과 변경사항을 정확하게 공유하는 것이 중요하다고 생각합니다. 특히 프론트엔드 개발은 기획 변경과 백엔드 API의 변화가 화면 구현에 직접적인 영향을 주기 때문에 필요한 정보를 빠르게 확인하고 팀원과 기준을 맞추는 데 신경 써왔습니다.

부트캠프에서 음악 플레이리스트 공유 SNS인 Mingle을 개발할 당시 6명으로 시작한 팀이 5명으로 줄고 개발 과정에서 기획과 API 명세가 여러 차례 변경되면서 빠른 소통이 더욱 중요해졌습니다. 매일 오전 데일리 스크럼을 통해 진행 상황과 개발 중 발생한 이슈를 공유했고 API 명세와 실제 응답값이 일치하지 않거나 필요한 데이터가 부족한 경우에는 백엔드 팀원과 즉시 소통하며 원인을 확인하고 문제를 해결했습니다. 변경된 내용 역시 팀원들과 빠르게 공유해 불필요한 재작업이 발생하지 않도록 했습니다.

외부 OpenWeatherMap API를 연동하는 과정에서 CORS Preflight 오류가 발생했습니다. 네트워크 요청과 OpenWeatherMap API 문서를 확인한 결과 내부 API용 공통 Axios 인스턴스의 Authorization Header가 외부 API 요청에도 전달되고 있는 것이 원인이었습니다. OpenWeatherMap 요청에서는 공통 Axios 인스턴스를 사용하지 않고 Authorization Header가 포함되지 않는 기본 Axios 요청으로 분리해 문제를 해결했습니다. 이후 원인과 해결 방법을 팀원들과 공유해 내부 인증 API와 외부 API의 요청 방식을 구분했습니다.

Git Flow를 기반으로 Milestone, Issue, Pull Request를 관리하고 코드 리뷰를 진행했습니다. 또한 네이밍, 함수 작성 방식, export 방식 등 팀의 코드 컨벤션을 맞춰 여러 프론트엔드 개발자가 작성한 코드의 일관성을 유지하며 협업했습니다.

이 경험을 통해 협업에서는 혼자 해결하는 것뿐 아니라 현재 상황과 변경사항을 팀이 같은 기준으로 이해할 수 있도록 공유하는 과정이 중요하다는 점을 배웠습니다. 입사 후에도 담당 업무의 진행 상황과 기술적인 이슈를 명확하게 공유하고, 동료들과 필요한 기준을 맞추며 안정적으로 서비스를 만들어가는 개발자가 되겠습니다.

## 지원 동기 및 입사 후 포부

[서비스의 흐름을 이해하고 지속적인 개선에 기여하는 개발자]

사용자의 행동과 운영 과정에서 발생하는 데이터를 실제 서비스 경험으로 연결하고 기능 구현 이후에도 지속적으로 개선하는 환경에서 프론트엔드 개발자로 기여하고 싶습니다. 여러 프로젝트를 직접 기획, 개발하며 실시간 데이터 처리, 운영 대시보드 구축, AI API 연동, 웹 성능 개선을 경험했고 개별 기능을 구현하는 것뿐 아니라 서비스 전체의 흐름을 이해하는 것이 중요하다는 점을 배웠습니다.

입사 후에는 먼저 서비스 구조와 사용자 흐름, 팀의 개발 방식을 빠르게 파악하겠습니다. 기획과 디자인, API 요구사항을 정확하게 이해하고 화면에서 필요한 데이터와 상태의 책임을 정리해 팀의 코드 기준에 맞는 기능을 안정적으로 구현하겠습니다. 기능 개발 이후에도 로딩 속도와 렌더링 과정, 예외 상황까지 점검하며 실제 사용 환경에서 발생하는 불편을 지속적으로 개선하겠습니다.

새로운 요구사항이나 기술적 문제가 발생했을 때 원인을 구체적으로 확인하고 해결 방향을 동료들과 공유하며 주어진 기능을 구현하는 것을 넘어 서비스의 개선점을 함께 고민하고 제안할 수 있는 프론트엔드 개발자로 자리 잡겠습니다. 또한 AI를 활용해 개발 효율과 문제 해결의 범위를 넓히고, 사용자가 서비스를 이용하는 맥락과 불편을 이해해 실제 사용자 경험 개선으로 연결하는 개발자로 성장하겠습니다.

## EDUCATION

### 한서대학교

항공소프트웨어공학과

02/2024

졸업

### 엘리스

웹 개발자 트랙 6기

08/2023 – 12/2023

수료

### 주요 학습 및 경험

- **프론트엔드 핵심역량 습득**
  - React 및 TypeScript 기반의 웹 개발 생태계 이해 및 상태 관리 기법 습득
- **실무형 애자일 협업 프로세스**
  - 데일리 스크럼 미팅, 백엔드 팀원과의 소통을 통한 팀 프로젝트 2건 완수
- **Git 기반 협업 체계**

- Issue 발행, 브랜치 전략(Git Flow) 및 Pull Request 기반 코드 리뷰 문화 적용

## CERTIFICATES

- 정보처리기사
- GTQ 1급